

# The Setup Section - Every Tool Explained

---

Setup is everything that gets defined BEFORE anything actually happens. Nothing here is Bob or Alice doing something - it is purely the toolbox they both reach into later. It splits into 6 groups.

## 1. THE THREE NUMBERS

n, q, k

## 2. BASIC POLY MATHS

add, sub, mult

## 3. THE NTT TOOLKIT

the fast multiply

## 4. SMALL RANDOM NUMBERS

CBD table

## 5. BIG RANDOM NUMBERS

rejection sampling

## 6. BYTE PACKING + COMPRESSION

getting ready to send

Groups 1-2 are the foundations. Group 3 replaced the slow multiply. Groups 4-5 build the random values everything is made from. Group 6 turns numbers into things you can actually send over a wire.

## Group 1: The Three Numbers

---

$n = 256$        $q = 3329$        $k = 4$

$n$  - how many coefficients live in each polynomial. Every polynomial in this whole project is a list of exactly 256 numbers.

$q$  - the modulus. Every single number stays between 0 and 3328. Anything that goes over wraps back round, which is what all the  $\% q$  you see everywhere is doing.

$k$  - the matrix and vector size.  $A$  is  $k \times k$  (so  $4 \times 4$ ), and  $S$ ,  $e$ ,  $b$ ,  $u$  are all  $k$  long.  
This one number is what makes it ML-KEM-1024 rather than 512 or 768.

## Group 2: Basic Polynomial Maths

---

### **poly\_add(f, g) and poly\_sub(f, g)**

WHAT: add or subtract two polynomials.

HOW: walk along both lists position by position, add (or subtract) the two numbers sitting at each position, wrap the answer with  $\% q$ . Nothing clever.

```
position 0:  f[0] + g[0]  mod q
position 1:  f[1] + g[1]  mod q
...and so on for all 256
```

### **poly\_mult(f, g) - the OLD slow way, no longer used**

WHAT: multiplies two polynomials the obvious schoolbook way.

HOW: two nested loops. Every number in  $f$  gets multiplied by every number in  $g$ , and each result is dropped into a bin decided by adding the two positions together.

That gives 511 slots, which is too many, so anything sitting at position 256 or higher gets folded back down and SUBTRACTED (because  $x^n = -1$  in this ring).

Cost:  $256 \times 256 =$  about 65,000 multiplications. This is why NTT exists.

It is kept in the file purely as a reference to check the NTT answer against.

## Group 3: The NTT Toolkit

---

This is what replaced `poly_mult`. Same answers, roughly 7x faster in practice.

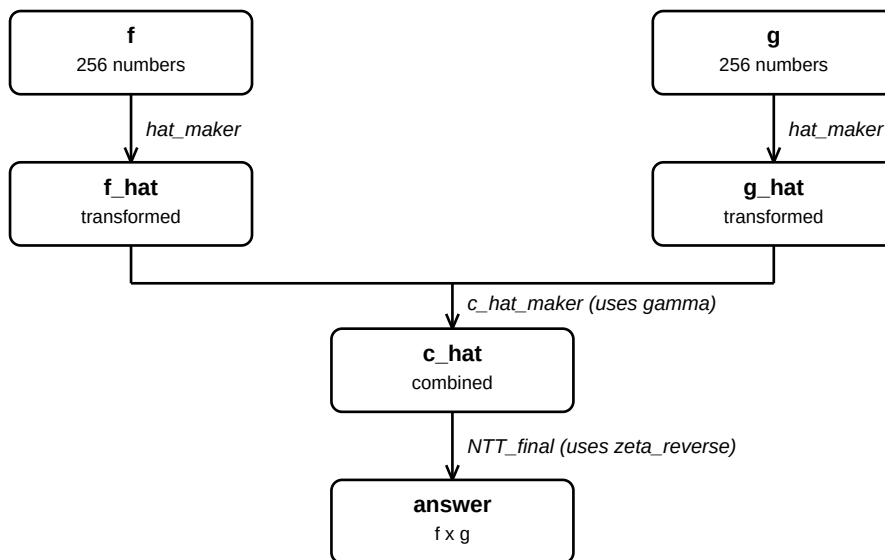
### The three lookup tables

`zeta` - 128 fixed numbers copied straight out of FIPS 203 Appendix A. NOT calculated by us. They are all powers of 17 mod 3329, but in a scrambled order (bit-reversal) that we deliberately take as given rather than derive.

`zeta_reverse` - that exact same list flipped end to end, so the reverse transform can walk forwards through it instead of having to walk `zeta` backwards.

`gamma` - a SEPARATE 128 numbers, also from Appendix A. Used only in the multiply step, never in the transform.

### The pipeline



## Group 3 continued: How each NTT function works

### **hat\_maker(f) - the forward transform**

WHAT: turns a normal polynomial into its transformed (hat) shape.

HOW: 7 rounds. Each round cuts the list into twice as many equal groups as the round before, and halves the pairing distance. Every group grabs the next unused zeta.

| Round | Groups | Zetas used | Pairing distance |
|-------|--------|------------|------------------|
| 1     | 1      | 1          | 128 apart        |
| 2     | 2      | 2          | 64 apart         |
| 3     | 4      | 4          | 32 apart         |
| 4     | 8      | 8          | 16 apart         |
| 5     | 16     | 16         | 8 apart          |
| 6     | 32     | 32         | 4 apart          |
| 7     | 64     | 64         | 2 apart          |

$1+2+4+8+16+32+64 = 127$ , so every zeta gets used exactly once.

The formula run on every single pair (a, b):

```
t      = zeta x b      mod q
new a  = a + t         mod q
new b  = a - t         mod q
```

That is called a butterfly. It happens 128 times per round, 7 rounds, so 896 times in total for one polynomial.

### **c\_hat\_coefficient and c\_hat\_maker - the multiply**

WHAT: multiplies two transformed polynomials together.

HOW: pairs here are NEIGHBOURS (position 0 with 1, 2 with 3...), not far apart like in hat\_maker. Every pair gets its OWN gamma - no sharing.

```
c0 = (a0 x b0) + (a1 x b1 x gamma)  mod q
c1 = (a0 x b1) + (a1 x b0)          mod q
```

c\_hat\_coefficient does one pair. c\_hat\_maker runs it across all 128 pairs.

### **NTT\_final(c\_hat) - the reverse transform**

WHAT: turns the transformed answer back into a normal polynomial.

HOW: same 7 rounds but backwards - pairing goes 2, 4, 8 ... up to 128. Reads zeta\_reverse forwards. The formula itself flips too:

```
new a = a + b          mod q
new b = zeta_reverse x (b - a) mod q
```

Then one last step: every number gets multiplied by 3303. That cancels out a leftover factor that builds up from doing 7 rounds.

### **ntt\_multiply(f, g)**

WHAT: bundles all of the above into one function.

HOW: hat\_maker(f), hat\_maker(g), c\_hat\_maker, NTT\_final. This is the ONLY one the rest of the code actually calls - it is a straight drop-in replacement for poly\_mult.

## Groups 4 and 5: Making Random Numbers

---

Two completely different jobs here, using two different tools.

### Group 4 - SMALL numbers, for secrets and noise (S, e, r, e\_one, e\_two)

These need to be TINY - only -2 to +2 - because they are the noise that makes the lattice problem hard while still letting the message decode correctly.

CBD\_TABLE - 256 entries, one for every possible byte value. Each entry gives back 2 small coefficients. Written out by hand rather than calculated.

```
byte 0    -> [ 0,  0]
byte 1    -> [ 1,  0]
byte 12   -> [-2,  0]
byte 255  -> [ 0,  0]
```

`poly_builder(poly_bytes, table)`

WHAT: turns a chunk of bytes into a full polynomial.

HOW: looks up every byte in the table and glues the pairs together end to end.

128 bytes in gives 256 coefficients out, which is exactly one polynomial.

### Group 5 - BIG numbers, for the matrix A

These need to be spread evenly across the whole 0 to 3328 range - the opposite of the small ones.

`poly_generator_from_seed(seed)`

WHAT: builds one cell of the matrix A from a seed.

HOW, step by step:

1. stretch the seed into 700 bytes using `shake_128`
2. read the stream 3 bytes at a time
3. each group of 3 bytes gives two 12-bit candidate numbers (d1 and d2)
4. keep any candidate under q, THROW AWAY anything 3329 or over
5. stop as soon as 256 coefficients have been kept

Step 4 is the important one - it is called rejection sampling. 12 bits can hold up to 4095, but we only want up to 3328. Simply wrapping the extras round with `% q` would make the low numbers slightly more likely than the high ones. Throwing them away instead keeps everything perfectly even.

## Group 6a: Byte Packing

---

Hash functions only accept bytes, and real implementations have to agree byte for byte. Python's `str()` would not do - it writes numbers out as readable TEXT, which wastes space and is Python-specific.

The number 1729 written two ways:

```
as text: "1729" = 4 characters = 32 bits
as bits: 11011000001 = 12 bits
```

Nearly 3x smaller, and completely unambiguous across any language.

### **bits\_to\_bytes and bytes\_to\_bits**

WHAT: convert between a list of individual bits and real bytes.

HOW: peel bits off one at a time using `% 2` (gives the bit) and `// 2` (moves along).

The same trick used in the CBD work early on.

### **byte\_encode(F, d)**

WHAT: packs a list of numbers into tightly packed real bytes, d bits each.

HOW: turn each number into d bits, glue every bit into one long list, pack into bytes.

Different d values used in this project:

```
d = 12 -> b and the raw polynomials (256 x 12 = 384 bytes)
d = 11 -> u after compression        (256 x 11 = 352 bytes)
d = 5  -> v after compression        (256 x 5 = 160 bytes)
d = 1  -> m, which is just bits      (256 x 1 = 32 bytes)
```

### **byte\_decode(B, d)**

WHAT: the exact reverse - unpacks bytes back into a list of numbers.

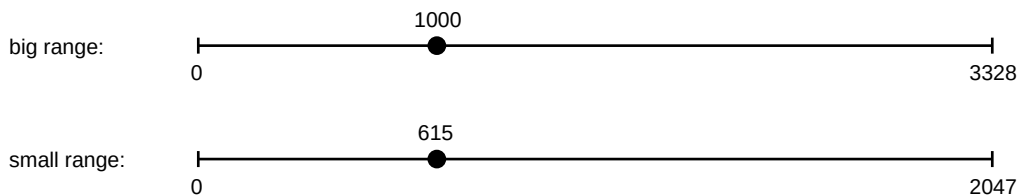
HOW: unpack to bits, read d bits at a time, rebuild each number.

## Group 6b: Compression

---

Compression deliberately throws accuracy away to save space. Unlike `byte_encode`, which is lossless, this one is lossy ON PURPOSE.

### The idea - the same position on a shorter ruler



1000 sits about 30% along the big range. 30% along the small range is 615.  
Same position, shorter ruler. That is the entire idea.

$$(1000 / 3328) \times 2047 = 615.08 \rightarrow \text{rounds to } 615$$

### The four functions

`compress_u` / `decompress_u` - always 11 bits (0 to 2047)

`compress_v` / `decompress_v` - always 5 bits (0 to 31)

HOW: the code does it with integer maths only, because FIPS 203 forbids floating point. The  $(a + b/2) // b$  pattern is just the integer way of rounding to nearest instead of always rounding down.

### Why v gets squeezed 6x harder than u

`v` only ever has to answer one rough question at the very end: closer to 0, or closer to halfway? Those two targets are miles apart, so even at 5 bits the answer is still obvious. Losing detail costs nothing.

`u` is different - it feeds into real ongoing algebra (multiplied by `S`, subtracted from `v`). Sloppiness there would snowball and could push the FINAL answer into the wrong half, breaking the very decision `v` is trying to make. So `u` stays more precise.

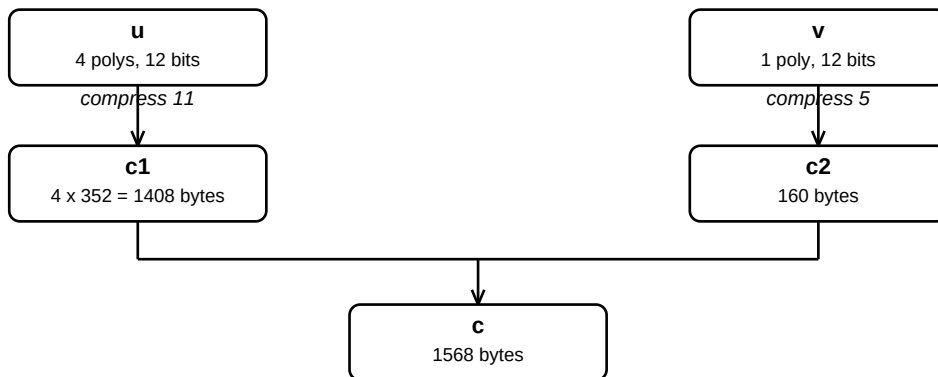
Measured error after a round trip: `u` is off by at most 1, `v` by at most 52.

## Group 6c: Building the Real Ciphertext

---

These two are what actually turn Alice's working numbers into something sendable.

### **ciphertext\_maker(u, v)**



$1408 + 160 = 1568$  bytes, which matches FIPS 203 Table 3 EXACTLY for ML-KEM-1024. This one blob is the only thing that travels from Alice to Bob.

### **ciphertext\_opener(c)**

WHAT: exactly the reverse, run by Bob the moment he receives **c**.

HOW: slice the first 1408 bytes off as **u** (in 352-byte chunks, one per poly), the remaining 160 as **v**. `byte_decode` each one, then decompress.

IMPORTANT: what Bob gets back is CLOSE to Alice's original **u** and **v** but NOT identical. That is the lossy compression showing up, and it is completely expected. The message still decodes correctly, which is the whole point.